

Module 8: Pattern Matching and Expression Bodies (optional)

Overview

- Pattern Matching Overview
- Switch Pattern Matching
- Is Pattern Matching
- Switch Pattern Expression and Expression Bodies

Pattern Matching (C# >=7.0)

= matching certain members in our objects or the objects themselves to certain conditions and values

- Easy to understand comparison code
- Matching
 - Any data type
 - Extracted value



switch Statements in C# <= 6.0

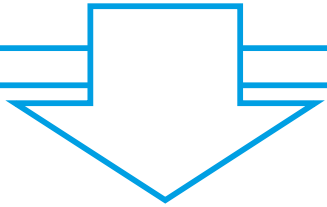
- You can use only
 - bool
 - char
 - string
 - numeric types like int and float
 - enums

```
void print(object o)
{
    switch (o) // not supported before C# 7.0
    {
        case typeof(Employee): // not supported at all
            toDo = "Make a payment"; break;
        default:
            toDo = "Nothing to do"; break;
    }
}
```



switch Statements in C# <= 6.0

```
switch (obj.GetType().Name)
{
    case "Employee":    // possible typos in code before C#6
        toDo = "Make a payment"; break;
    default:
        toDo = "Nothing to do"; break;
}
```



```
switch (obj.GetType().Name)
{
    case nameof(Employee):    // nameof() expression from C#6
        toDo = "Make a payment"; break;
    default:
        toDo = "Nothing to do"; break;
}
```

Note:

- *Nameof is evaluated at compile time, has no effect at runtime!*

Type Pattern Matching in Switch (C# 7.0)

- Switch statement use type patterns to check an object's runtime type
- Using a discard (_,), we match the type without needing to capture the object.

```
switch(o)    // C# 7.0
{
    case Employee _: // Type pattern with discard (_)
        toDo = "Make a payment"; break;
    case Customer _:
        toDo = "Send a newsletter"; break;
    default:
        toDo = "Nothing to do"; break;
}
```

Type Pattern Matching with Variable Binding

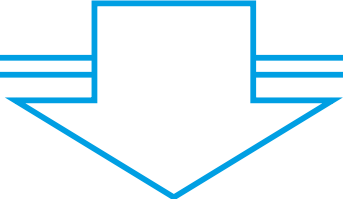
- Type patterns in switch statements to both check an object's type and capture it in a variable
- Benefit: Cleaner, safer alternative to manual type casting and is checks

```
switch (o)
{
    case Employee e:
        toDo = $"Pay salary to {e.Name}"; break;
    case Customer c:
        toDo = $"Send newsletter to {c.Email}"; break;
    default:
        toDo = "Nothing to do"; break;
}
```

switch Pattern Matching (C# 7.0)

```
switch (o)
{
    case int i:    // Type Pattern
        Console.WriteLine($"Int is: {i}"); break;
    case DateTime d:
        Console.WriteLine($"Date is: {d.ToLocalTime()}"); break;
    default:
        Console.WriteLine("Not match");
        break;
}
```

SYNTACTIC SUGAR



```
if (o is int)
{
    int num = (int)o;
    Console.WriteLine(string.Format("Int is: {0}", num));
}
else if (o is DateTime)
{
    DateTime dateTime = (DateTime)o;
    Console.WriteLine(string.Format("Date is: {0}", dateTime));
}
else
{
    Console.WriteLine("Not match");
}
```

when Statement in case Clause (C# 7.0)

```
switch (o)
{
    case int i when (i == 0):
        Console.WriteLine($"Int is Zero"); break;
    case int i when ((i > 0) && (i < 10)):
        Console.WriteLine($"Int is greater than Zero and lower than Ten"); break;
    default:
        Console.WriteLine("Not match");
        break;
}
```

SYNTACTIC SUGAR

```
if (o is int)
{
    int num = (int)o;
    if (num == 0)
    {
        Console.WriteLine("Int is Zero");
        return;
    }
    int num2 = num;
    if (num2 > 0 && num2 < 10)
    {
        Console.WriteLine("Int is greater than Zero and lower than Ten");
        return;
    }
}
Console.WriteLine("Not match");
```

null Statement in case Clause

- Matching null value

```
object o = null;
switch (o)
{
    case int i when (i == 0):
        Console.WriteLine($"Int is Zero"); break;
    case int i when ((i > 0) && (i < 10)):
        Console.WriteLine($"Int is greater than Zero and lower than Ten");
        break;
    case null:
        Console.WriteLine($"Is null"); break;
    default:
        Console.WriteLine("Not match");
        break;
}
```

is Pattern Matching C# 7.0

- Variable has local scope not for if but for method

```
static void Print(object o)
{
    if (o is string) // Declaration Pattern
    {
        string s = o as string;
        Console.WriteLine($"UpperCase: {s.ToUpper()}");
    }
}
```

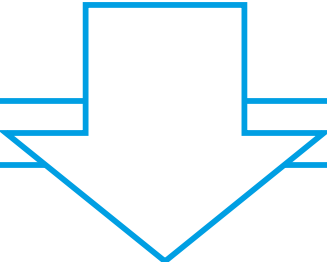
```
static void Print(object o)
{
    if (o is string s) // Declaration Pattern
    {
        Console.WriteLine($"UpperCase: {s.ToUpper()}");
    }
}
```

SYNTACTIC SUGAR

is Pattern Matching C# 7.0

- Variable has local scope not for if but for method

```
static void Print(object o)
{
    if (o is string s) // Declaration Pattern
    {
        Console.WriteLine($"UpperCase: {s.ToUpper()}");
    }
}
```

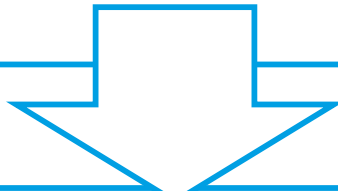


```
static void Print(object o)
{
    string s = o as string;
    if (s != null)
    {
        Console.WriteLine(string.Concat("UpperCase: ", s.ToUpper()));
    }
}
```

Local Variable is created

- Beware of duplicat names in same scope

```
if (o is string s) { Console.WriteLine($"UpperCase: {s.ToUpper()}"); }  
if (o is sbyte s) { Console.WriteLine($"Value: {s}"); } // s is already defined
```



```
if (o is string s1) { Console.WriteLine($"UpperCase: {s1.ToUpper()}"); }  
if (o is sbyte s2) { Console.WriteLine($"Value: {s2}"); }
```

Lab 08.01



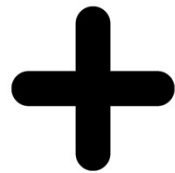
Exercise:

- Is Pattern Matching
- Switch Pattern Matching



New language Pattern Matching Features

- C# 7.0: Matching Constant Pattern
- C# 8.0: Matching Property, Tuple and Positional Pattern
- C# 9.0: Matching Logical, Relational and Var Pattern



- C# 8.0: Switch Expressions



switch Expression in C# 8.0

- switch pattern

```
switch (obj)
{
    case Employee _:
        toDo = "Make a payment"; break;
    default:
        toDo = "Nothing to do"; break;
}
```

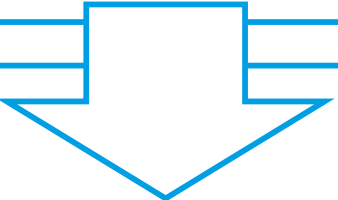
- switch expression C# 8

```
string toDo = obj switch
{
    Employee => "Make a payment",
    _ => "Nothing to do"
};
```



return Value using switch Expression

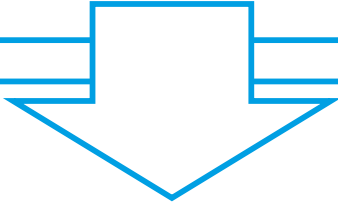
```
static string GetToDoString(object obj)
{
    string toDo = "";
    switch (obj)
    {
        case Employee _:
            toDo = "Make a payment"; break;
        default:
            toDo = "Nothing to do"; break;
    }
    return toDo;
}
```



```
static string GetToDoString(object obj)
{
    return obj switch
    {
        Employee => "Make a payment",
        _ => "Nothing to do"
    };
}
```

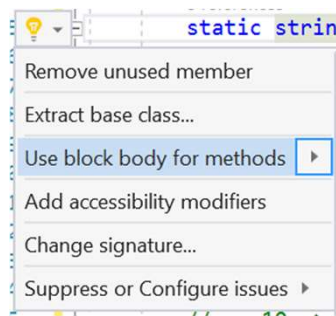
Expression Body Method

```
static string GetToDoString(object obj)
{
    return obj switch
    {
        Employee => "Make a payment",
        _ => "Nothing to do"
    };
}
```



```
static string GetToDoString(object obj) => obj switch
{
    Employee => "Make a payment",
    _ => "Nothing to do"
};
```

Tip: Visual Studio Quick Actions



Handling all possible values

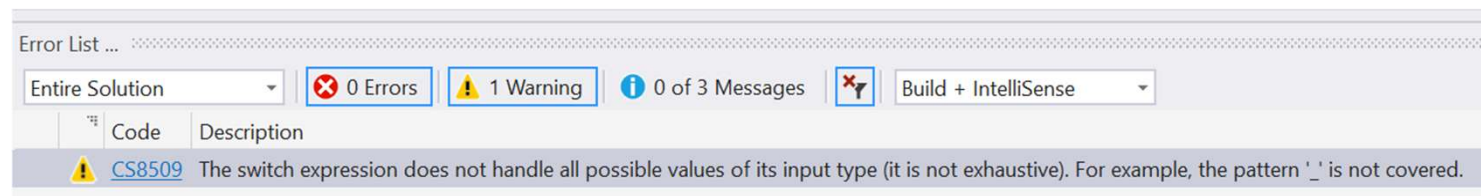
- switch expression does not handle all possible values

```
static string GetToDoString(object obj) => obj switch
{
    string => "Make a payment",
    // _ => "Nothing to do"    // missing default not every value is covered
};
```

```
static string GetToDoString(object obj)
{
    if (obj is string)
    {
        return "Make a payment";
    }
    throw new SwitchExpressionException(obj);
}
```

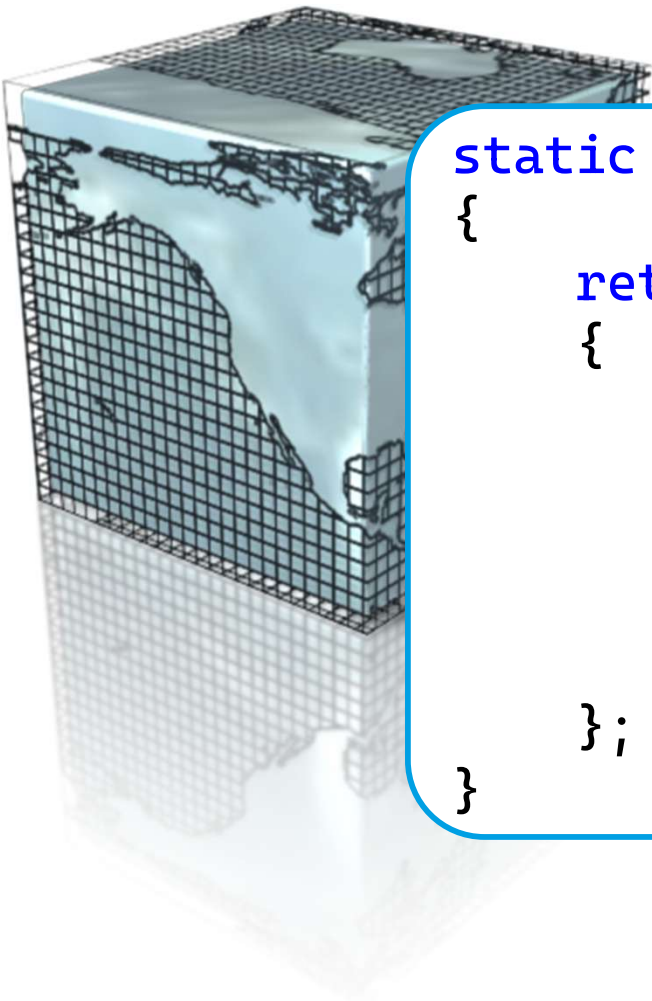
Note:

- It is compiled showing warning, runtime error will be thrown when uncovered value will be passed*



Exercise - switch Expression Body

- Refactor to switch expression body



```
static string CategorizeTemperature(int temp)
{
    return temp switch
    {
        < 0 => "Freezing",
        <= 15 => "Cold",
        <= 25 => "Mild",
        <= 35 => "Warm",
        _ => "Hot"
    };
}
```

Temperature -5°C is Freezing.
Temperature 10°C is Cold.
Temperature 20°C is Mild.
Temperature 30°C is Warm.
Temperature 40°C is Hot.

Overview

- Pattern Matching Overview
- Switch Pattern Matching
- Is Pattern Matching
- Switch Pattern Expression and Expression Bodies